Sir Syed CASE Institute of Technology

Islamabad

WEB ENGINEERING

Lab 02

Semantic HTML and Accessibility Foundations

Lab Report

SUBMITTED BY

Muhammad Hamza

ROLL NUMBER

2330-0068

DATE

11 October 2026

Page structure, keyboard access and accessibility audit evidence

Semantic page structure

Tasks 1 and 2 | Skip link and landmarks

TASK QUESTION

Add a Skip to main content link as the first focusable element. Place the existing greeting and list navigation inside a header. Wrap the content in main with id="main-content" and add a footer containing address.

The Lab 1 greeting is preserved. The page now has a header, navigation, one main landmark and a footer. The heading order is one h1 followed by an h2 for Office Hours and an h2 for Send Feedback.

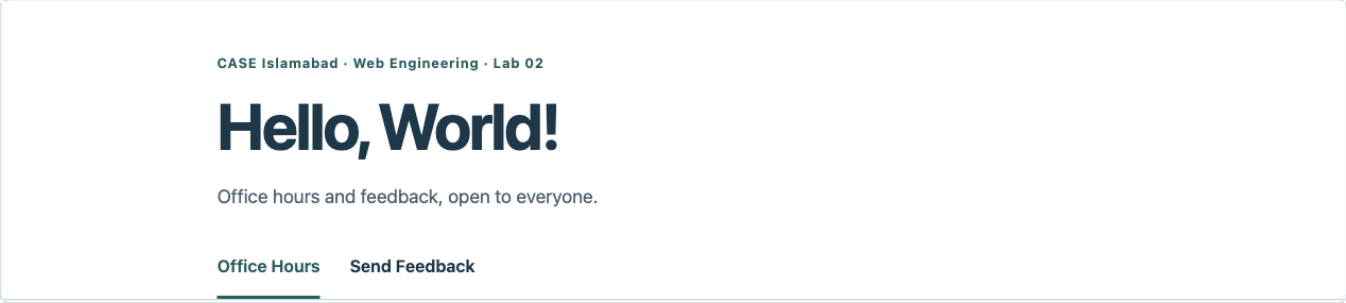


Figure 1. The greeting and section navigation inside the page header.

LANDMARK STRUCTURE

```
<a href="#main-content" class="skip-link">Skip to main content</a>
<header>
  <h1 id="greeting">Hello, World!</h1>
  <nav><ul>
    <li><a href="#schedule">Office Hours</a></li>
    <li><a href="#feedback">Send Feedback</a></li>
  </ul></nav>
</header>
<main id="main-content" tabindex="-1"> ... </main>
<footer><address>Contact your instructor at CASE Islamabad.</address></footer>
```

The skip link points to the main landmark. Its tabindex value allows focus to move to main when the link is activated, without adding main to the normal Tab sequence.

Office hours and SVGs

Tasks 3 and 4 | Table structure and image meaning

TASK QUESTION

Add an Office Hours section with an h2, caption, thead, tbody, scoped column headers and time elements. Add a decorative SVG divider and an informative SVG inside figure and figcaption, with role="img" and a title.

The table presents sample office hours in three columns. Each time has a datetime value. Row and column headers identify the data. The location pin is named for assistive technology; the divider is decorative and hidden.

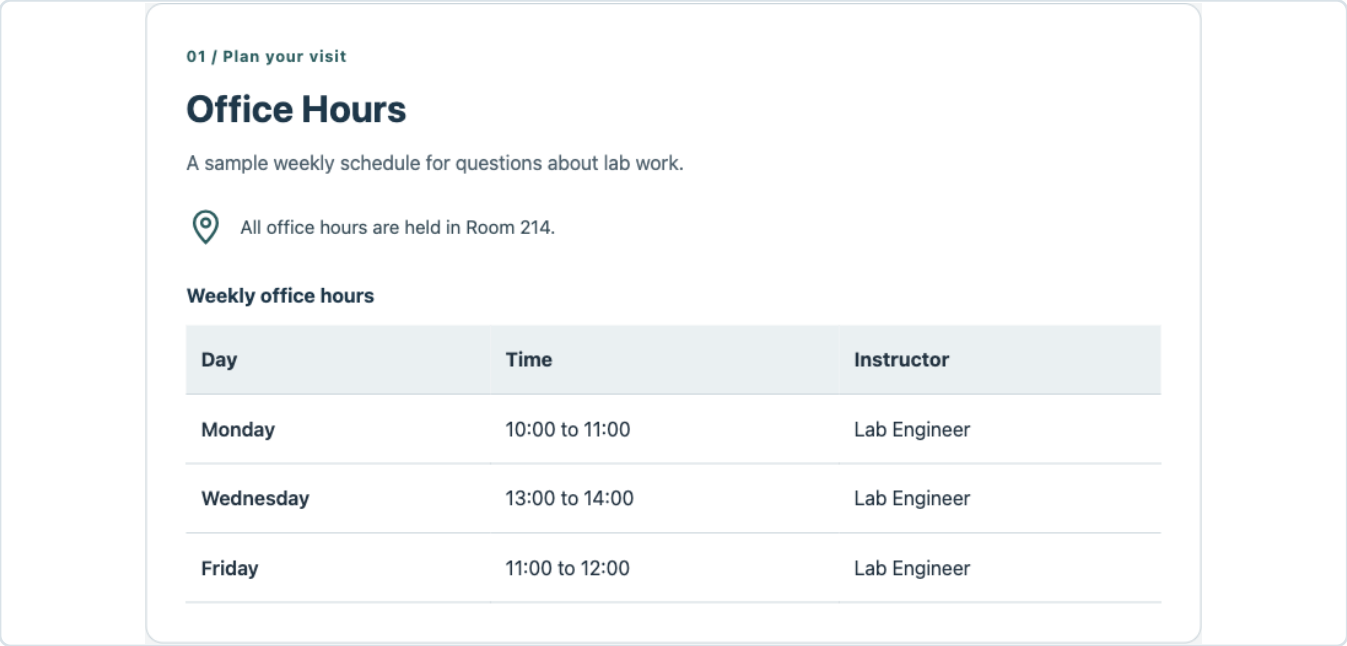


Figure 2. Office Hours table and the informative location pin.

ACCESSIBILITY ATTRIBUTES

```
<th scope="col">Day</th>
<th scope="col">Time</th>
<th scope="col">Instructor</th>
<time datetime="10:00">10:00</time>
<svg aria-hidden="true" focusable="false"> ... </svg>
<figure>
  <svg role="img" aria-labelledby="pin-title">
    <title id="pin-title">Location pin</title> ...
  </svg>
  <figcaption>All office hours are held in Room 214.</figcaption>
</figure>
```

Accessible feedback form

Tasks 5 and 6 | Labels and grouped controls

TASK QUESTION

Add a Send Feedback section with an h2 and labelled name and email fields. Add a Topic radio group using fieldset and legend, a labelled message textarea and a submit button.

Every input has a label connected through matching for and id values. The Topic options share a fieldset and legend. The form uses native inputs, radio buttons, a textarea and a submit button.

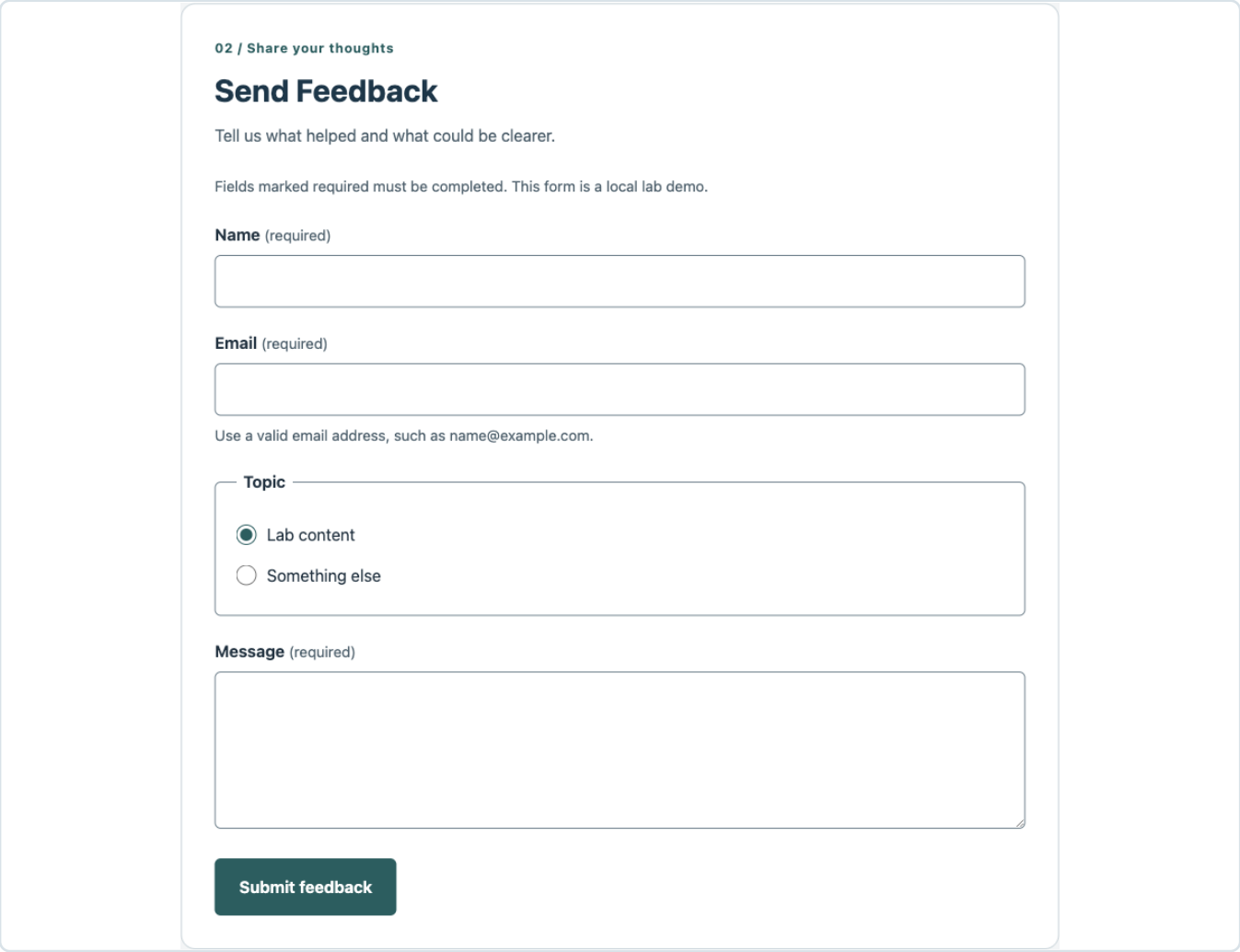


Figure 3. The completed feedback form with visible labels and a Topic group.

The form is a local demonstration. A valid submission displays a status message without sending feedback to a server. The status region announces the result without moving the keyboard focus.

Required fields and ARIA

Tasks 7 and 9 | Validation, descriptions and current location

TASK QUESTION

Mark required fields with required and connect the email hint with aria-describedby. Add aria-current="location" to the navigation link for the section in view. Remove redundant ARIA.

Name, email and message are required. Their labels also state this in visible text. The browser checks missing values and email format. The email hint is connected to the field so its explanation is available to assistive technology.

EMAIL FIELD AND HINT

```
<label for="email">Email (required)</label>
<input type="email" id="email" name="email"
       autocomplete="email" required
       aria-describedby="email-hint" />
<p id="email-hint">Use a valid email address,
  such as name@example.com.</p>
```

Control	Validation or description
Name	Required text field with a matching label
Email	Required email field with an associated hint
Message	Required textarea with a matching label

CURRENT NAVIGATION LINK

```
<a href="#schedule" aria-current="location">Office Hours</a>
<a href="#feedback">Send Feedback</a>
```

The script moves aria-current to Send Feedback when that section enters view and returns it to Office Hours at the top of the page. Exactly one link has the current location attribute. No redundant roles are added to the landmarks.

Accessibility tree check: The informative SVG is exposed as an image named Location pin. The decorative divider is hidden with aria-hidden. The heading and landmark structure uses native HTML.

Keyboard access

Task 8 | Focus visibility and traversal

TASK QUESTION

Add a visible :focus-visible style and show the skip link when focused. Test Tab, Shift + Tab, Enter and Space across the page. Confirm the expected order and no keyboard trap.

The eight Tab stops were checked in both directions. Enter activates the skip link and the submit button. Arrow keys move between Topic options, and Space selects a radio option. An empty submission focuses the required Name field.

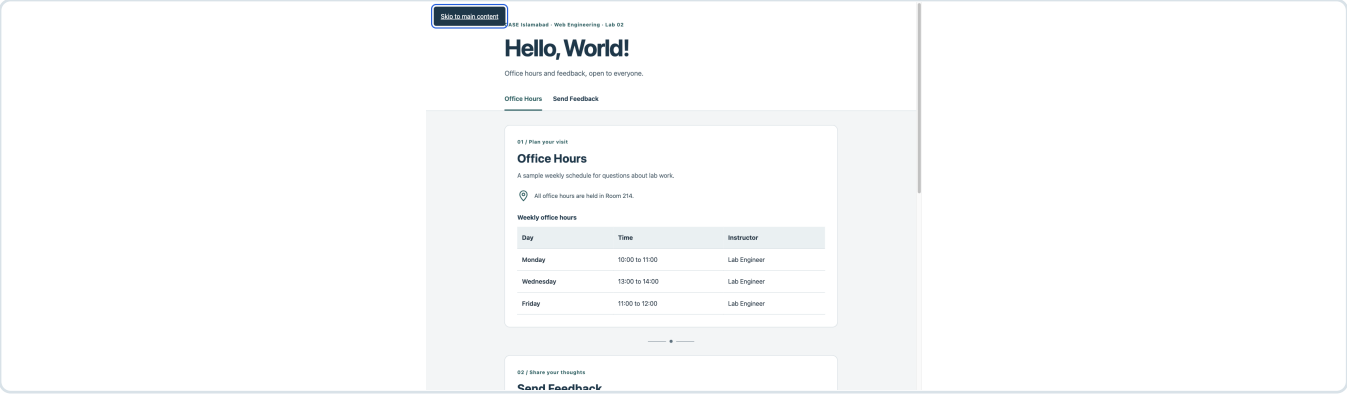


Figure 4. The first Tab reveals the skip link and its blue focus outline.

Stop	Control	Stop	Control
1	Skip to main content	5	Email
2	Office Hours	6	Topic radio group
3	Send Feedback	7	Message
4	Name	8	Submit feedback

```
:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 4px;
}
.skip-link { transform: translateY(-160%); }
.skip-link:focus { transform: translateY(0); }
```

axe DevTools scan

Task 10 | WCAG 2.1 AA audit

TASK QUESTION

Install the axe DevTools browser extension, scan the page and fix all reported accessibility violations.

The axe DevTools extension was installed in a dedicated Chrome profile and used to scan the page served through Live Server. The scan reports zero automatic issues, with zero issues in every severity category.

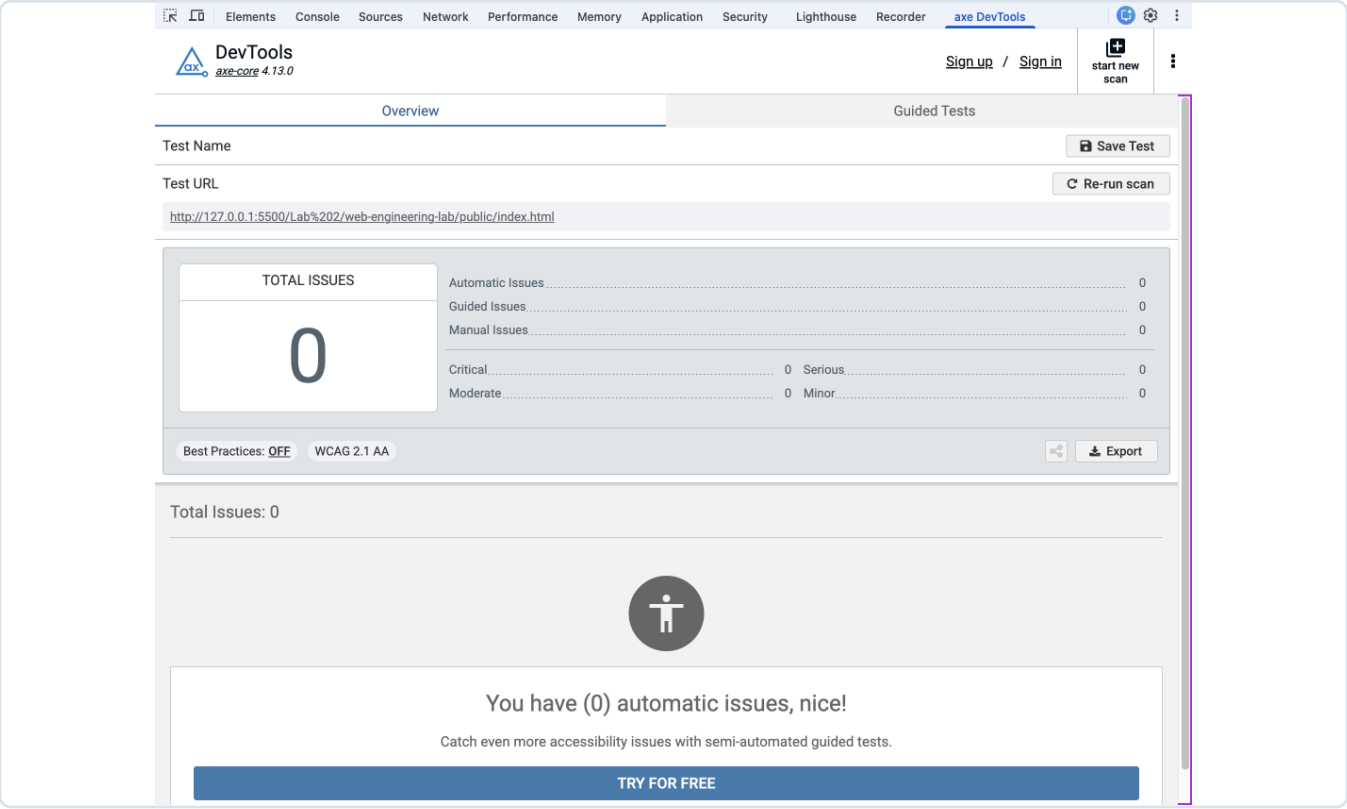


Figure 5. The actual axe DevTools extension scan shows Total Issues: 0.

Severity	Reported issues
Critical and Serious	0
Moderate and Minor	0

A separate axe engine check also reported zero violations and 48 passing checks. The keyboard checks were completed separately. No paid guided test was used.

Lighthouse accessibility

Task 11 | Chrome DevTools audit

TASK QUESTION

Run a Lighthouse Accessibility audit in Chrome DevTools. Fix remaining issues until the score is 100 and save a screenshot of the report.

Lighthouse was run in Chrome DevTools with Accessibility selected. The page scored 100 out of 100. The report screenshot includes the inspected Live Server address and the Accessibility score.

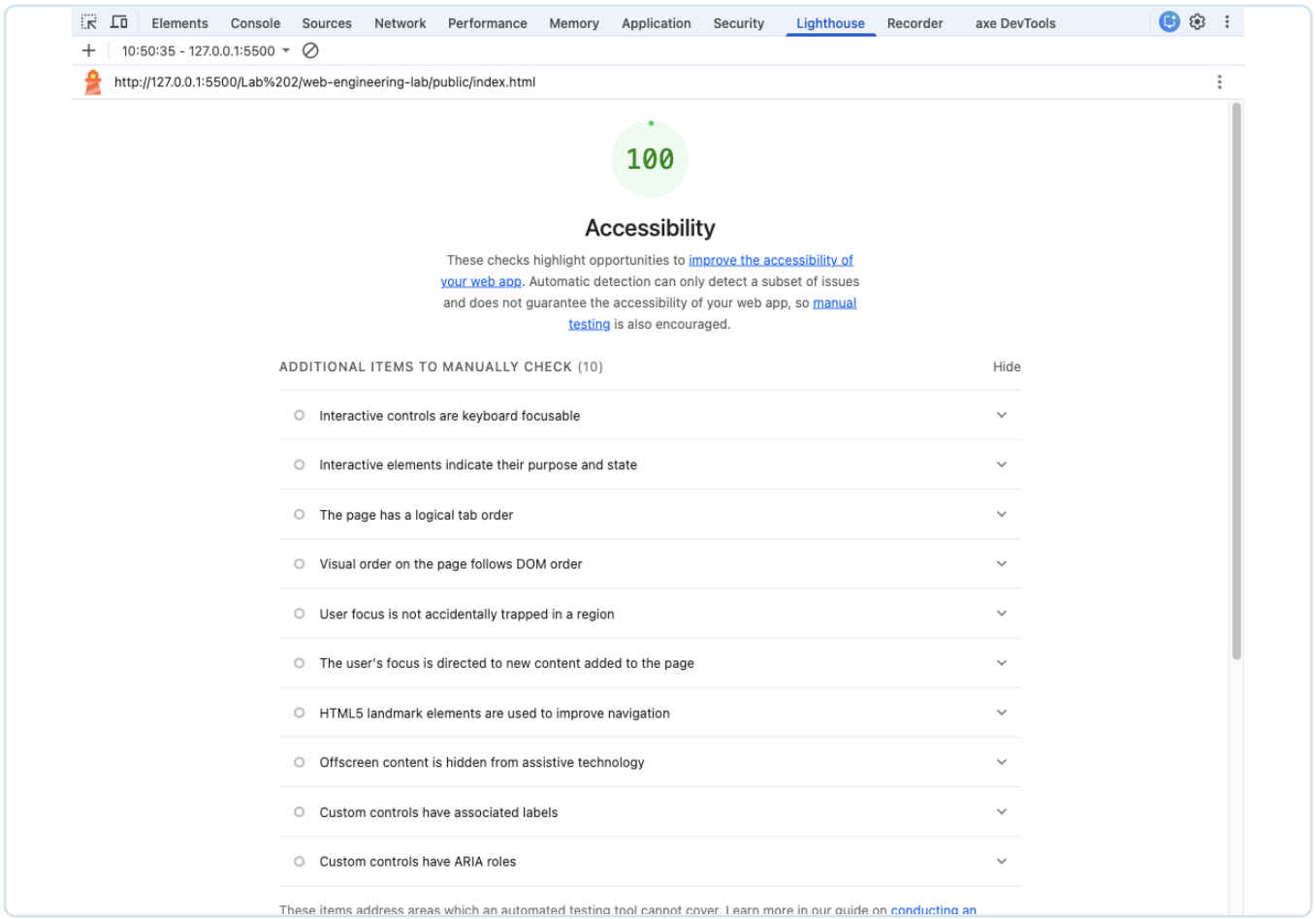


Figure 6. Chrome DevTools Lighthouse report with an Accessibility score of 100.

Saved evidence: lighthouse-devtools.png contains the browser audit screenshot. A separate Lighthouse command line run also scored 100 and saved the complete HTML and JSON reports.

The keyboard checks confirm the tab order, visible focus and lack of a focus trap. Automated scores cover a subset of accessibility checks; they do not replace every manual review.

GitHub submission

Task 12 | Commit and push

TASK QUESTION

Commit all changes to the web-engineering-lab repository from Lab 1 and push the updated project to GitHub.

Lab 02 uses the same repository and retains the Lab 1 commit history. The updated page, stylesheet, script and evidence were committed on main and pushed to GitHub. A separate local checkout is saved inside the Lab 2 folder.

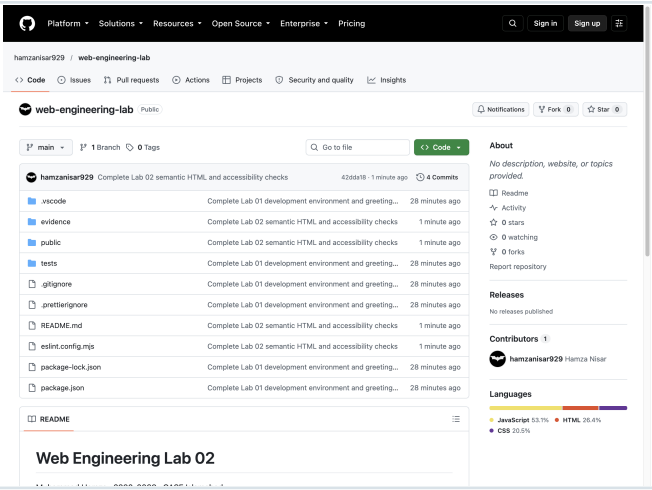


Figure 7. The GitHub repository after the Lab 02 changes were pushed.

PUSH CONFIRMATION

```
To github.com:hamzanisar929/web-engineering-lab.git
   c5a9497..42dda18  main -> main
branch 'main' set up to track 'origin/main'.
```

Repository: https://github.com/hamzanisar929/web-engineering-lab

Evidence folder: evidence/lab02 contains the page screenshots, axe results, Lighthouse reports, keyboard checks and Git submission output.

REFERENCES

Lab 02 Statement, Semantic HTML and Accessibility Foundations, Sir Syed CASE Institute of Technology, Islamabad. Task questions are restated from the supplied handout. CASE logo from the Lab 01 handout.

Deque axe DevTools documentation · Google Lighthouse documentation